

Monitoreo de Temperatura - Humedad - ACC

Jeimy Tatiana Palacios Torres
Sara Osorio Castaño
Electrónica digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	4
5. Estructura del código	5
6. Explicación de funciones principales	7
7. Comunicación (si aplica)	9
8. Procedimiento de prueba	9
9. Manejo de errores y seguridad	10

1. Identificación del proyecto

- **Nombre del proyecto:** Monitoreo de Temperatura - Humedad - ACC
- **Integrantes del equipo:** Jeimy Tatiana Palacios Torres, Sara Osorio Castaño

2. Resumen

Este proyecto implementa un sistema biomédico IoT utilizando una placa ESP32 programada en MicroPython y un servidor web desarrollado en Flask. El sistema permite el monitoreo en tiempo real de variables ambientales y de movimiento mediante la integración de sensores de temperatura, humedad y aceleración, junto con mecanismos de alerta y supervisión remota.

El dispositivo emplea un sensor DHT11 para la medición de temperatura y humedad, y un acelerómetro MPU6050 para detectar movimientos leves y bruscos. Además, incorpora un buzzer para la generación de alertas sonoras y un botón de pánico que permite activar notificaciones de emergencia inmediatas. El sistema analiza continuamente los datos obtenidos y clasifica el estado del entorno en diferentes condiciones, tales como temperatura alta, humedad baja, movimiento anómalo o estado normal.

La comunicación remota se realiza mediante un bot de Telegram, el cual permite consultar información del sistema a través de comandos específicos, incluyendo temperatura, humedad, estado del movimiento y umbrales configurados. Asimismo, el sistema envía notificaciones automáticas cuando se detectan eventos anómalos o situaciones de riesgo.

Complementariamente, se desarrolló un servidor web utilizando Flask, encargado de recibir y almacenar los datos enviados por la ESP32 mediante solicitudes HTTP POST. El servidor incorpora una interfaz gráfica interactiva desarrollada con TailwindCSS y Chart.js, la cual permite visualizar en tiempo real los valores de temperatura y humedad, el estado del sistema, gráficas dinámicas y un historial de registros recientes.

El proyecto demuestra la integración de tecnologías IoT, monitoreo remoto y visualización web en un sistema biomédico de bajo costo, ofreciendo una solución funcional para supervisión ambiental y detección de eventos críticos en tiempo real.

3. Descripción del hardware

- **ESP32** fue utilizada como unidad principal de procesamiento y control del sistema biomédico IoT, programada en MicroPython. Esta placa se encarga de:
- Leer los sensores de temperatura, humedad y movimiento.
- Gestionar la comunicación WiFi.
- Controlar el buzzer de alertas.
- Detectar eventos del botón de pánico.

- Enviar información al servidor Flask y al bot de Telegram.
Además, se utilizaron diferentes pines GPIO para la conexión de sensores y dispositivos de salida.
- **Sensor DHT11** Utilizado para medir la temperatura y humedad ambiental del entorno monitoreado.
- Pin de datos: GPIO 4
- **Variables medidas:** Temperatura en °C, humedad relativa
- **Sensor MPU6050** Sensor acelerómetro y giroscopio utilizado para detectar movimiento y cambios bruscos de aceleración del sistema. El módulo se comunica mediante protocolo I2C.
Funciones: Detección de movimiento leve, Detección de movimiento brusco, Identificación de eventos anómalos.
SCL al Pin 22
SDA al Pin 21
- **Botón de Pánico:** al Pin 19
Se implementó un botón físico de emergencia para activar alertas inmediatas. Al ser presionado, el sistema genera una alarma sonora y envía una notificación automática mediante Telegram.
- **Buzzer Pasivo** al Pin 25
El buzzer fue utilizado como sistema de alarma sonora del dispositivo
Funciones : Alertas leves para movimiento moderado, Alertas fuertes para (movimiento brusco, temperatura extrema, humedad extrema, activación de botón de pánico)
- **Servidor Local Flask:** El sistema incluye un servidor web desarrollado en Flask que permite:
 - Recibir datos desde la ESP32 mediante HTTP POST
 - Almacenar temporalmente registros
 - Mostrar información en tiempo real
 - Generar gráficas dinámicas de temperatura y humedad
 - Visualizar estados y eventos del sistema
- **Comunicación Inalámbrica:** La comunicación del sistema se realiza mediante conexión WiFi integrada en la ESP32. **Funciones :** Envío de datos al servidor Flask, Comunicación con el bot de Telegram, Supervisión remota del sistema.
- **Fuente de Alimentación:**El sistema es alimentado mediante conexión USB de 5V para energizar

4. Descripción del software

- **Lenguaje usado:** MicroPython

- **Librerías utilizadas en la ESP32:**

machine: (para la configuración y control de pines GPIO, PWM (buzzer) y comunicación I2C (sensor MPU6050)),

time: (para la gestión de temporización, retardos y control del tiempo de ejecución del ciclo principal),

network: (Permite la conexión de la ESP32 a la red WiFi para la transmisión de datos y comunicación remota)

math: Utilizada para realizar cálculos matemáticos relacionados con la magnitud del movimiento detectado por el acelerómetro.

dht: Permite la lectura de temperatura y humedad del sensor DHT11.

thread: Usada para ejecutar el sistema de escucha del bot de Telegram en un hilo independiente, permitiendo comunicación simultánea con el monitoreo de sensores.

urequests: Utilizada para enviar datos al servidor Flask mediante solicitudes HTTP POST. **utelegram:** Permite la comunicación entre la ESP32 y el bot de Telegram para envío y recepción de mensajes. **mpu6050:** Librería empleada para la lectura de datos del acelerómetro MPU6050

- **Librerías utilizadas en el servidor Flask:**

flask: Framework utilizado para desarrollar el servidor web y las rutas de comunicación HTTP. **collections.deque:** Permite almacenar un historial limitado de registros del sistema. **time:** Utilizada para generar marcas de tiempo asociadas a cada registro recibido. **tailwindCSS:** Framework CSS utilizado para el diseño visual de la interfaz web **Chart.js:** Librería JavaScript utilizada para generar gráficas dinámicas en tiempo real

- **IDE:** Thonny, Visual Studio Code

Flujo general del programa:

1. Inicialización del sistema y configuración de los pines GPIO utilizados para sensores, buzzer, botón de pánico y comunicación I2C con el acelerómetro MPU6050.
2. Conexión automática de la ESP32 a la red WiFi utilizando las credenciales configuradas previamente
3. Inicialización del bot de Telegram y envío de un mensaje de confirmación indicando que el sistema biomédico se encuentra conectado y operativo.
4. Configuración de los sensores DHT11 y MPU6050 para iniciar la lectura de temperatura, humedad y movimiento.
5. Creación de un hilo independiente para mantener activa la escucha de mensajes y comandos enviados desde Telegram sin interrumpir el monitoreo de sensores

6. Inicio del ciclo principal del sistema, donde se realizan continuamente lecturas de temperatura y humedad mediante el sensor DHT11.
7. Lectura de aceleración en los ejes X, Y y Z mediante el sensor MPU6050 y cálculo de la magnitud del movimiento detectado.
8. Comparación de los valores obtenidos con los umbrales definidos para identificar: temperatura, humedad, movimiento
9. Activación de alertas sonoras utilizando PWM en el buzzer cuando se detectan condiciones críticas o movimientos bruscos
10. Detección de activación del botón de pánico y envío inmediato de notificaciones de emergencia mediante Telegram.
11. Procesamiento de comandos enviados por el usuario desde Telegram para consultar información como: Temperatura actual, Humedad actual, Estado del movimiento, Estado general del sistema , Umbrales configurados
12. Envío periódico de los datos monitoreados al servidor Flask mediante solicitudes HTTP POST
13. Recepción y almacenamiento temporal de los datos en el servidor web.
14. Actualización automática de la interfaz gráfica web
15. Ejecución continua del sistema cada 5 segundos para mantener el monitoreo y la supervisión en tiempo real.

5. Estructura del código

- Archivo principal: `main.py` El archivo principal del sistema corresponde a `main.py`, ejecutado directamente en la ESP32. Este archivo contiene toda la lógica relacionada con: Conexión Wifi, comunicación con Telegram, lectura de sensores, detección de eventos anómalos, activación de alertas, envío de datos al servidor Flask. Además, se importan librerías externas necesarias para el funcionamiento de sensores y servicios de comunicación, como `mpu6050` y `uttelegram`.
- Archivo del servidor: `servidor.py` El sistema también incluye un segundo archivo ejecutado en un computador o servidor local denominado `servidor.py`, desarrollado en Python utilizando Flask. Este archivo se encarga de:
 - Recibir datos enviados desde la ESP32
 - Almacenar temporalmente los registros
 - Proporcionar una API REST

- Mostrar una interfaz gráfica web en tiempo real
- El sistema está estructurado alrededor de un ciclo principal infinito (`while True`) ejecutado en la ESP32, encargado del monitoreo continuo de sensores y procesamiento de eventos en tiempo real.

Dentro de este ciclo principal se realizan: Lectura de sensores, verificación de sensores, gestión de alertas, comunicación con telegram, envío de datos al servidor

- **Variables Globales** El programa utiliza variables globales para almacenar y actualizar continuamente la información obtenida por los sensores y el estado general del sistema. Entre estas variables se encuentran la temperatura, la humedad, el estado del movimiento detectado, el estado actual del sistema, el último mensaje recibido desde Telegram y el último estado enviado como notificación. También se emplea una variable para controlar la activación del botón de pánico. El uso de variables globales permite compartir información entre las diferentes funciones del programa y mantener una actualización constante de los datos monitoreados.
- **Modularización mediante Funciones** El código se encuentra organizado en diferentes funciones con el propósito de separar tareas específicas y facilitar la lectura, mantenimiento y escalabilidad del sistema. Cada función cumple una responsabilidad concreta dentro del funcionamiento general del programa.

La función `calcular-movimiento()` se encarga de obtener los valores de aceleración del sensor MPU6050 y calcular la magnitud promedio del movimiento detectado. Esto permite clasificar si el sistema se encuentra en reposo, movimiento leve o movimiento brusco.

La función `activar-buzzer()` controla el buzzer mediante señales PWM para generar alertas sonoras. Dependiendo del tipo de evento detectado, el sistema puede emitir alertas leves o fuertes, diferenciando situaciones normales de eventos críticos.

La función `enviar-servidor()` permite transmitir los datos monitoreados hacia el servidor Flask utilizando solicitudes HTTP POST. La información enviada incluye temperatura, humedad, movimiento y estado general del sistema.

La función `procesar-estados()` analiza continuamente los valores obtenidos de los sensores y los compara con los umbrales establecidos. A partir de este análisis, el sistema determina estados como temperatura alta, humedad baja, movimiento brusco o evento anómalo, activando alarmas y notificaciones cuando es necesario.

La función `procesar-comandos()` administra los mensajes enviados desde Telegram. Esta función interpreta los comandos escritos por el usuario y responde automáticamente con información relacionada con el estado actual del sistema biomédico.

Finalmente, la función `telegram-thread()` mantiene activo el proceso de escucha de mensajes del bot de Telegram en un hilo independiente. Esto permite que la ESP32 continúe monitoreando sensores y ejecutando tareas principales mientras recibe comandos en tiempo real.

- **Estructura del Servidor Flask** El archivo `servidor.py` está organizado utilizando rutas proporcionadas por Flask para gestionar la comunicación entre la ESP32 y la

interfaz web. La ruta principal (/) se encarga de mostrar la página web del sistema biomédico, incluyendo las tarjetas informativas, gráficas dinámicas y tabla de registros históricos.

La ruta /datos recibe la información enviada desde la ESP32 mediante solicitudes HTTP POST. Cada registro recibido contiene datos de temperatura, humedad, movimiento y estado general, junto con una marca temporal que permite organizar cronológicamente la información.

Por otra parte, la ruta /api/datos funciona como una API REST que entrega los registros almacenados en formato JSON. Esta información es utilizada por el frontend para actualizar automáticamente las gráficas, tablas y estados mostrados en la interfaz web en tiempo real.

6. Explicación de funciones principales

- **Pin ()**: Permite configurar los pines GPIO de la ESP32 como entradas o salidas digitales. En el sistema biomédico se utiliza para conectar el sensor DHT11, el botón de pánico y el buzzer, permitiendo controlar la comunicación entre la ESP32 y los diferentes dispositivos electrónicos.
- **PWM()**: Se utiliza para generar señales moduladas en el buzzer. Gracias a esta técnica, el sistema puede producir diferentes tipos de sonidos y alertas dependiendo del evento detectado, diferenciando entre alarmas leves y alarmas críticas.
- **I2C()**: Gestiona la comunicación serial entre la ESP32 y el sensor MPU6050 mediante el protocolo I2C. Esta comunicación permite obtener los valores de aceleración utilizados para detectar movimiento y eventos bruscos.
- **network.WLAN()**: La función `network.WLAN()` permite configurar y administrar la conexión WiFi de la ESP32. Mediante esta función el sistema se conecta a la red inalámbrica para enviar datos al servidor Flask y comunicarse con Telegram.
- **bot.send()**: es utilizada para enviar mensajes automáticos mediante Telegram. El sistema emplea esta función para: confirmar conexión, reportar estados anómalos, enviar alertas de emergencia, responder comandos del usuario.
- **bot.listen()**: Mantiene activo el bot de Telegram para recibir mensajes y comandos enviados por el usuario en tiempo real.
- **measure()**: Pertenece al sensor DHT11 y permite actualizar las mediciones de temperatura y humedad antes de leer sus valores.
- **temperature() y humidity()**: Estas funciones permiten obtener los valores de temperatura y humedad capturados por el sensor DHT11. Los datos obtenidos son utilizados para evaluar el estado ambiental del sistema.

- **get-accel()**:pertenece a la librería del MPU6050 y permite obtener los valores de aceleración en los ejes X, Y y Z. Estos datos son utilizados para calcular la magnitud del movimiento detectado.
- **calcular-movimiento()**:Esta función calcula la magnitud promedio del movimiento a partir de varias lecturas del acelerómetro. El sistema utiliza este cálculo para clasificar el estado del movimiento en:reposo, movimiento leve, movimiento brusco.
- **activar-buzzer()** Controla las alertas sonoras del sistema. Dependiendo del parámetro recibido, puede generar: alertas leves para movimientos moderados, alertas fuertes para eventos críticos o emergencias.
- **enviar-servidor()**: Esta función se encarga de enviar los datos monitoreados hacia el servidor Flask mediante solicitudes HTTP POST. Los datos enviados incluyen temperatura, humedad, estado del movimiento y estado general del sistema.
- **procesar-estados()**:Analiza continuamente los valores obtenidos por los sensores y los compara con los umbrales establecidos. A partir de este análisis, determina el estado actual del sistema y activa alarmas o notificaciones cuando detecta condiciones anómalas.
- **procesar-comandos()**:Esta función administra los mensajes enviados desde Telegram. Dependiendo del comando recibido, el sistema responde automáticamente con información específica del monitoreo, como temperatura, humedad, movimiento o estado general.
- **telegram-thread()**: mantiene activo el proceso de escucha del bot de Telegram en un hilo independiente. Esto permite que el sistema continúe ejecutando el monitoreo en tiempo real sin interrumpir la recepción de mensajes.
- **request.json**: permite recibir los datos enviados desde la ESP32 en formato JSON. Estos datos son procesados y almacenados temporalmente para su visualización.
- **deque()**: se utiliza para almacenar un historial limitado de registros recibidos desde la ESP32. Esto permite mantener únicamente los datos más recientes y optimizar el uso de memoria del servidor
- **jsonify()**:convierte los registros almacenados en formato JSON para que puedan ser enviados al frontend y visualizados dinámicamente en la página web.
- **fetch()**:Utilizada en JavaScript permite solicitar periódicamente los datos al servidor Flask desde la interfaz web. Gracias a esto, las gráficas y tablas se actualizan automáticamente en tiempo real.
- **Chart()**: Pertenece a la librería Chart.js y permite crear gráficas dinámicas para visualizar la evolución de la temperatura y humedad del sistema biomédico

- Lógica de control del sistema: La lógica general del sistema coordina el monitoreo biomédico, la detección de anomalías, la generación de alertas y la comunicación remota. El programa ejecuta continuamente la lectura de sensores, el procesamiento de estados y la actualización de la interfaz web, permitiendo supervisión en tiempo real y respuesta inmediata ante eventos críticos

7. Comunicación (si aplica)

El proyecto implementa comunicación inalámbrica mediante la conectividad WiFi integrada en la placa ESP32, permitiendo la transmisión de datos en tiempo real hacia un servidor local y la interacción remota mediante Telegram.

La ESP32 se conecta a una red inalámbrica utilizando el módulo `network.WLAN()`, lo que permite establecer comunicación con el servidor Flask desarrollado en Python. A través de solicitudes HTTP POST, el sistema envía periódicamente información relacionada con: temperatura, humedad, estado del movimiento, estado general del sistema .

El servidor Flask recibe y almacena temporalmente los datos enviados desde la ESP32, permitiendo su visualización en una interfaz web dinámica accesible desde cualquier navegador conectado a la red local. La interfaz muestra gráficas en tiempo real, estados del sistema y un historial de registros recientes.

Además, el sistema incorpora comunicación remota mediante un bot de Telegram, el cual permite enviar notificaciones automáticas y recibir comandos del usuario en tiempo real. Gracias a esta funcionalidad, el usuario puede consultar información del sistema desde cualquier dispositivo con acceso a Telegram.

La comunicación entre la ESP32 y Telegram se realiza utilizando internet mediante la API del bot, permitiendo supervisión remota continua del sistema biomédico.

El sistema también utiliza comunicación interna mediante protocolo I2C entre la ESP32 y el sensor MPU6050. Esta comunicación permite obtener datos de aceleración en tiempo real para detectar movimiento y eventos bruscos.

La arquitectura implementada permite ampliar fácilmente el sistema para incorporar nuevas funcionalidades, como: almacenamiento en base de datos, monitoreo en la nube, aplicaciones móviles, paneles de control avanzados, integración con otros dispositivos IoT, sistemas de alerta médica remota.

8. Procedimiento de prueba

1. Conectar la placa ESP32 al computador mediante cable USB y verificar la correcta alimentación del sistema.
2. Cargar y ejecutar el archivo `main.py` en la ESP32 utilizando el entorno de desarrollo Thonny.
3. Ejecutar el archivo `servidor.py` en el computador para iniciar el servidor Flask y habilitar la interfaz web del sistema biomédico.

4. Verificar que la ESP32 se conecte correctamente a la red WiFi configurada y comprobar en la consola la dirección IP asignada.
5. Confirmar la recepción del mensaje inicial enviado por el bot de Telegram indicando que el sistema biomédico se encuentra conectado.
6. Acceder desde un navegador web a la dirección IP del servidor Flask y comprobar la correcta visualización de: tarjetas informativas, gráficas dinámicas, tabla de registros históricos
7. Verificar el funcionamiento del sensor DHT11 comprobando que los valores de temperatura y humedad se actualicen correctamente tanto en la consola como en la interfaz web.
8. Mover físicamente el sistema para comprobar la lectura del sensor MPU6050 y validar la detección de: movimiento lento, movimiento brusco,
9. Confirmar que el buzzer genere alertas sonoras diferentes dependiendo del tipo de evento detectado
10. Simular condiciones ambientales fuera de los umbrales establecidos para verificar la detección de: alta temperatura, baja temperatura, alta humedad, baja humedad
11. Verificar que al detectarse un estado anómalo el sistema: active el buzzer, Actualice el estado en la interfaz web, Envíe una notificación automática mediante Telegram.
12. Presionar el botón de pánico conectado al GPIO 19 para comprobar: Activación inmediata de la alarma sonora, Envío automático de mensaje de emergencia por Telegram
13. Enviar comandos desde Telegram para validar la comunicación remota del sistema
14. Confirmar que la ESP32 envíe correctamente los datos al servidor Flask mediante solicitudes HTTP POST.
15. Verificar que las gráficas de temperatura y humedad se actualicen automáticamente en tiempo real dentro de la interfaz web.
16. Revisar la tabla de registros históricos para comprobar el almacenamiento temporal y visualización cronológica de los datos recibidos
17. Mantener el sistema en funcionamiento continuo durante varios minutos para validar la estabilidad de la comunicación, el monitoreo y la actualización en tiempo real.

9. Manejo de errores y seguridad

- El sistema implementa mecanismos básicos de control y validación para garantizar un funcionamiento estable y seguro durante la ejecución continua del monitoreo biomédico. La ESP32 realiza verificaciones constantes de conexión WiFi y manejo de excepciones mediante estructuras try-except, evitando que errores de lectura o fallos temporales interrumpan completamente el funcionamiento del programa.

- Durante la adquisición de datos, el sistema valida continuamente las lecturas provenientes de los sensores DHT11 y MPU6050. En caso de presentarse errores en la comunicación o fallos de lectura, el programa captura la excepción y muestra el error en consola, permitiendo que el sistema continúe operando sin detener el ciclo principal.
- El botón de pánico utiliza interrupciones por hardware (IRQ-FALLING), permitiendo detectar eventos de emergencia de manera inmediata y eficiente sin necesidad de consultar constantemente el estado del botón dentro del ciclo principal. Esta implementación mejora la velocidad de respuesta ante situaciones críticas.
- El sistema incorpora umbrales de seguridad para temperatura, humedad y movimiento. Cuando los valores medidos superan los límites establecidos, el programa activa automáticamente alertas sonoras y notificaciones remotas, permitiendo una respuesta rápida ante condiciones anómalas.
- El buzzer es controlado mediante señales PWM, permitiendo activar y desactivar las alarmas de manera controlada. Después de cada alerta, el sistema deshabilita automáticamente la señal PWM para evitar sonidos continuos no deseados o sobrecarga del dispositivo.
- En el servidor Flask, los datos recibidos desde la ESP32 son almacenados utilizando una estructura deque con tamaño limitado (maxlen), evitando el crecimiento excesivo de memoria y manteniendo únicamente los registros más recientes del sistema.
- La interfaz web actualiza la información de manera periódica mediante solicitudes controladas (fetch), evitando recargas completas de la página y reduciendo el consumo de recursos del servidor.
- La comunicación entre la ESP32 y el servidor se realiza dentro de una red local, disminuyendo riesgos de acceso externo no autorizado durante las pruebas. Asimismo, la comunicación con Telegram utiliza la API oficial del servicio para enviar notificaciones y responder comandos remotos.
- El sistema opera utilizando los niveles lógicos de 3.3V propios de la ESP32, garantizando compatibilidad y seguridad eléctrica en los pines GPIO utilizados para sensores, buzzer y botón de pánico.
- Finalmente, la arquitectura modular del programa facilita el mantenimiento y la detección de errores, permitiendo ampliar el sistema con nuevas funciones de seguridad, almacenamiento o monitoreo remoto en futuras versiones.